Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине «ОМО»

Выполнил:
Студент 3-го курса
Группы АС-68
Жуков А.А.
Проверил:
Крощенко А.А.

Брест 2026

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Ход работы

Вариант 2 Диагностика рака груди

- Breast Cancer Wisconsin
- **Задача:** определить, является ли опухоль злокачественной (бинарная классификация).
- **Архитектура:**
  - входной слой;
  - один скрытый слой с 16 нейронами (ReLU);
  - выходной слой с 1 нейроном (Sigmoid).
- **Эксперимент:** обучите модель с 32 нейронами в скрытом слое. Как изменились метрики precision и recall?

Код программы

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score)

torch.manual_seed(42)
np.random.seed(42)

# датасет
df = pd.read_csv("breast_cancer.csv")

df = df.drop(columns=[c for c in df.columns
                      if "unnamed" in c.lower() or c.lower() == "id"],
             errors="ignore")
df = df.dropna()

col = "target" if "target" in df.columns else "diagnosis"
y = df[col]
if y.iloc[0] in ("M", "B"):
    y = y.map({"M": 0, "B": 1})        # 0 = злокачественная, 1 =
доброкачественная
y = y.values.astype(np.int64)

X = df.drop(columns=[col]).values.astype(np.float32)

print(f"Объектов: {X.shape[0]}, признаков: {X.shape[1]}")
print(f"Баланс классов (0=malignant, 1=benign): {np.bincount(y)}")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)
```

```
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test  = torch.tensor(X_test,  dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
y_test  = torch.tensor(y_test,  dtype=torch.float32).view(-1, 1)

N_FEATURES = X_train.shape[1]

# архитектура модели
class MLP(nn.Module):
    def __init__(self, n_features, hidden=16):
        super().__init__()
        self.fc1 = nn.Linear(n_features, hidden)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden, 1)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

# модель
def build_model(hidden, lr=0.001):
    model = MLP(N_FEATURES, hidden=hidden)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    return model, criterion, optimizer

# обучение
def train_model(hidden, epochs=200, lr=0.001):
    model, criterion, optimizer = build_model(hidden, lr)

    for epoch in range(epochs):
        model.train()
        y_pred = model(X_train)
        loss = criterion(y_pred, y_train)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"  Обучение завершено (hidden={hidden}, финальный
loss={loss.item():.4f})")
    return model

# оценка
def evaluate_model(model, name):
    model.eval()
    with torch.no_grad():
        logits = model(X_test)
        probs = torch.sigmoid(logits).numpy().ravel()

    y_pred = (probs >= 0.5).astype(int)
    y_true = y_test.numpy().ravel().astype(int)

    acc  = accuracy_score(y_true, y_pred)
    prec_0 = precision_score(y_true, y_pred, pos_label=0, zero_division=0)
    rec_0  = recall_score(y_true, y_pred, pos_label=0, zero_division=0)
    f1_0   = f1_score(y_true, y_pred, pos_label=0, zero_division=0)

    print(f"\n===== {name} =====")
    print(f"Accuracy:            {acc:.4f}")
    print(f"Precision (malignant): {prec_0:.4f}")
    print(f"Recall    (malignant): {rec_0:.4f}")
```

```
    print(f"F1        (malignant): {f1_0:.4f}")

    return {"accuracy": acc, "precision": prec_0,
            "recall": rec_0, "f1": f1_0}

# 16 vs 32
print("\n>>> Модель A: 16 нейронов")
model_16 = train_model(hidden=16)
res_16 = evaluate_model(model_16, "Модель A: 16 нейронов")

print("\n>>> Модель B: 32 нейрона")
model_32 = train_model(hidden=32)
res_32 = evaluate_model(model_32, "Модель B: 32 нейрона")

# сравнение
print("\n================ СРАВНЕНИЕ ================")
print(f"{'Метрика':<14}{'16 нейронов':>14}{'32 нейрона':>14}{'Δ':>10}")
print("-" * 52)
for k, title in [("accuracy", "Accuracy"),
                 ("precision", "Precision"),
                 ("recall", "Recall"),
                 ("f1", "F1")]:
    a, b = res_16[k], res_32[k]
    print(f"{title:<14}{a:>14.4f}{b:>14.4f}{b - a:>+10.4f}")

# вывод эксперимента
d_prec = res_32["precision"] - res_16["precision"]
d_rec  = res_32["recall"]    - res_16["recall"]
EPS = 0.02

print(f"\nВывод: при увеличении скрытого слоя с 16 до 32 нейронов "
      f"precision по классу malignant изменился на {d_prec*100:+.2f} п.п., "
      f"recall — на {d_rec*100:+.2f} п.п. "
      f"{'Изменения в пределах шума.' if abs(d_prec) < EPS and abs(d_rec) <
EPS else 'Изменения значимы.'}")
```

Вывод программы
Объектов: 569, признаков: 30
Баланс классов (0=malignant, 1=benign): [212 357]

>>> Модель A: 16 нейронов
  Обучение завершено (hidden=16, финальный loss=0.1084)

===== Модель A: 16 нейронов =====
Accuracy: 0.9649
Precision (malignant): 0.9318
Recall   (malignant): 0.9762
F1   (malignant): 0.9535

>>> Модель B: 32 нейрона
  Обучение завершено (hidden=32, финальный loss=0.0850)

===== Модель B: 32 нейрона =====
Accuracy: 0.9561
Precision (malignant): 0.9302
Recall   (malignant): 0.9524

F1 (malignant): 0.9412

========== СРАВНЕНИЕ ==========

Метрика	16 нейронов	32 нейрона	Δ
Accuracy	0.9649	0.9561	-0.0088
Precision	0.9318	0.9302	-0.0016
Recall	0.9762	0.9524	-0.0238
F1	0.9535	0.9412	-0.0123

Вывод: при увеличении скрытого слоя с 16 до 32 нейронов precision по классу malignant изменился на -0.16 п.п., recall — на -2.38 п.п. Изменения значимы.

Вывод: освоил построение, обучение и оценку MLP для бинарной классификации.

В эксперименте сравнил 16 и 32 нейрона в скрытом слое. Модель с 16 нейронами: accuracy 0.9649, recall (malignant) 0.9762. Увеличение до 32 нейронов ухудшило результат (accuracy 0.9561, recall 0.9524, −2.38 п.п.), при более низком train loss (0.0850 против 0.1084) — признак переобучения.